

VIT®
CHENNAI
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Name: Yashmit Sharma

Reg. No.: 24BAI1348

Software Engineering DA-2

Goal

Turn your requirements (user stories & wireframes) into a clean, thoughtful design. Show you understand modularity, cohesion, coupling, abstraction, and how to structure your code/UI properly.

1. Software Design Document with Code Snippets

a. Design Principles you Applied

Overview: In HabitForge, software engineering principles were rigorously engineered into both the Spring Boot backend and the React Vite frontend to ensure high maintainability, testability, and resilience to change.

i. Abstraction (Hiding Implementation Details)

Abstraction separates operational contracts from concrete execution. In HabitForge, abstraction is applied at two levels:
1. Interface Contracts: Business operations are defined through interfaces like HabitService, HabitRepository, and StreakStrategy. Callers interact exclusively through these interfaces, making the persistence engine completely swappable.
2. Data Transfer Objects (DTOs): Internal database models are hidden behind HabitRequestDto and HabitResponseDto, guaranteeing that API clients cannot spoof internal streak states.

```
// HabitService.java - Business Logic Abstraction Contract
package com.example.HabitForge.service;

public interface HabitService {
    HabitResponseDto createHabit(String userId, HabitRequestDto request);
    HabitResponseDto completeHabit(String habitId, String userId);
    List<HabitResponseDto> getUserHabits(String userId);
    void deleteHabit(String habitId);
}

// StreakStrategy.java - Strategy Pattern Abstraction
public interface StreakStrategy {
    int calculateNewStreak(Habit habit, LocalDate completionDate);
    boolean isStreakBroken(Habit habit, LocalDate currentDate);
}
```

ii. Modularity (Autonomous, Reusable Components)

Modularity partitions the system into self-contained units with explicit interfaces:
? Backend: Organized by layers: controller, service, strategy, repository, and dto/model.
? Frontend: Structured into focused UI components (QuestBoard, HeroStats, HabitCard) and an isolated API client service (habitService.js).

```
// frontend/src/services/habitService.js - Modular API Client
const API_BASE = '/api';

export const habitService = {
  async getHabits(userId = 'user-default') {
    const res = await fetch(`${API_BASE}/habits?userId=${userId}`);
    if (!res.ok) throw new Error('Failed to fetch quests');
    return res.json();
  },

  async completeHabit(habitId, userId = 'user-default') {
    const res = await fetch(`${API_BASE}/habits/${habitId}/complete?userId=${userId}`, { method: 'POST' });
    return res.json();
  }
};
```

iii. High Cohesion (Single Responsibility Principle)

Cohesion measures how focused a class is on a single well-defined task:
? StreakCalculationService: Exclusively computes consecutive days, evaluating streak breaks, and midnight resets without touching HTTP or password logic.
? GamificationService: Dedicated strictly to RPG mechanics?health point (HP) deductions, experience (XP) curves, and gold coins.
? HabitController: Exclusively handles HTTP request mapping, validation, and status codes.

```
// StreakCalculationService.java - Highly Cohesive Domain Service
@Service
public class StreakCalculationService {
    private final StreakStrategy streakStrategy;

    @Autowired
    public StreakCalculationService(StreakStrategy streakStrategy) {
        this.streakStrategy = streakStrategy;
    }

    public void processHabitCompletion(Habit habit, LocalDate date) {
        int updatedStreak = streakStrategy.calculateNewStreak(habit, date);
        habit.setCurrentStreak(updatedStreak);
        if (updatedStreak > habit.getLongestStreak()) {
            habit.setLongestStreak(updatedStreak);
        }
        habit.setCompletedToday(true);
        habit.setLastCompletedDate(date);
    }
}
```

iv. Low Coupling (Inversion of Control & Loose Dependencies)

Coupling describes how interdependent modules are. HabitForge achieves low coupling via:
? Spring Constructor Dependency Injection: Collaborators are injected as abstractions rather than hardcoded with 'new'.
? Stateless REST Contracts: Frontend and backend share zero binary dependencies; they communicate exclusively via standard JSON over HTTP.

```
// HabitServiceImpl.java - Loosely Coupled via Constructor Dependency Injection
@Service
public class HabitServiceImpl implements HabitService {
    private final HabitRepository habitRepository;
    private final StreakCalculationService streakCalculationService;
    private final GamificationService gamificationService;

    @Autowired
    public HabitServiceImpl(HabitRepository habitRepository,
                            StreakCalculationService streakCalculationService,
                            GamificationService gamificationService) {
        this.habitRepository = habitRepository;
        this.streakCalculationService = streakCalculationService;
        this.gamificationService = gamificationService;
    }

    @Override
    public HabitResponseDto completeHabit(String habitId, String userId) {
        Habit habit = habitRepository.findById(habitId).orElseThrow();
        if (!habit.isCompletedToday()) {
            streakCalculationService.processHabitCompletion(habit, LocalDate.now());
            habitRepository.save(habit);
            gamificationService.awardHabitRewards(userId, habit.getXpReward());
        }
        return mapToDto(habit);
    }
}
```

b. High-Level Architecture

HabitForge implements a 3-Tier Layered Client-Server Architecture orchestrated locally via Docker Compose across ports 5173 (React SPA), 8080 (Spring Boot REST API), and 27017 (MongoDB).

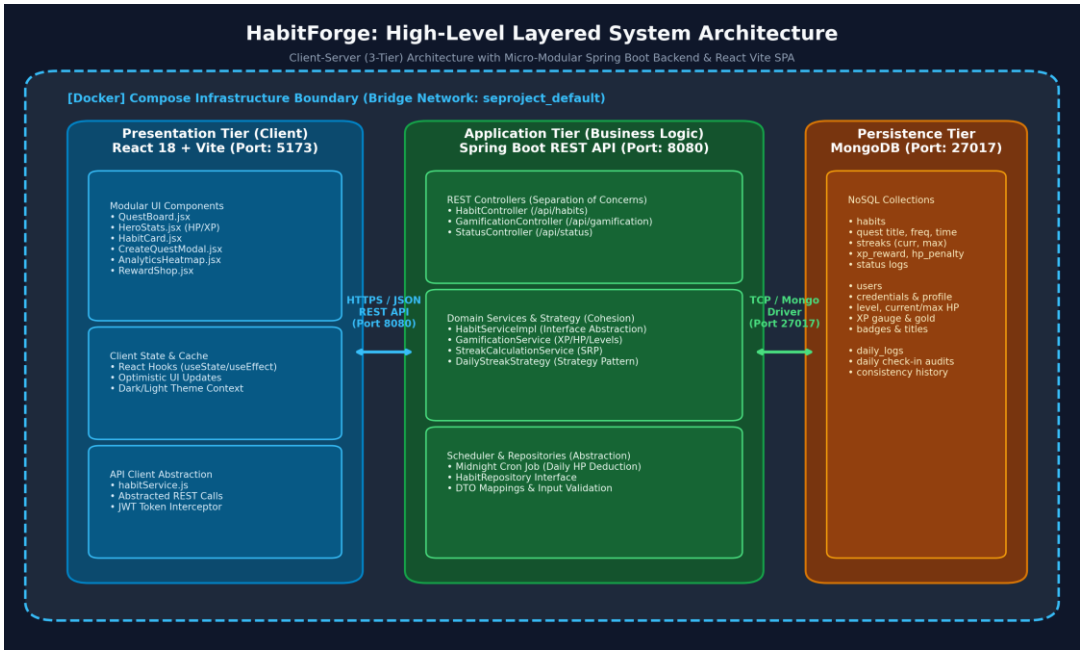


Figure 1: HabitForge High-Level Layered Architecture Diagram (Draw.io)

What Style Did You Choose & Why?

We chose the 3-Tier Layered Client-Server Architecture over monolithic server-rendered MVC or microservices for four core engineering reasons:

1. Strict Separation of Concerns: Presentation (React) manages UI state without risking business integrity. Application Tier (Spring Boot) enforces streak rules and security, while Persistence (MongoDB) stores unstructured JSON documents.
2. Independent Scalability & Deployment: Static frontend assets can be globally cached via edge CDNs, while the Spring Boot backend container scales independently.
3. Multi-Client & Mobile Adaptability: The backend exposes standard stateless RESTful JSON endpoints, so developing a future native mobile application (Flutter / React Native) requires zero backend changes.
4. Docker Containerization: Developers run the complete stack locally using 'docker-compose up --build', ensuring 100% environment parity between development and production.

Component Modularity Flow & UML Class Modeling

Below are the component interaction data flow diagram and UML class diagram illustrating interface abstraction and the Strategy Pattern:

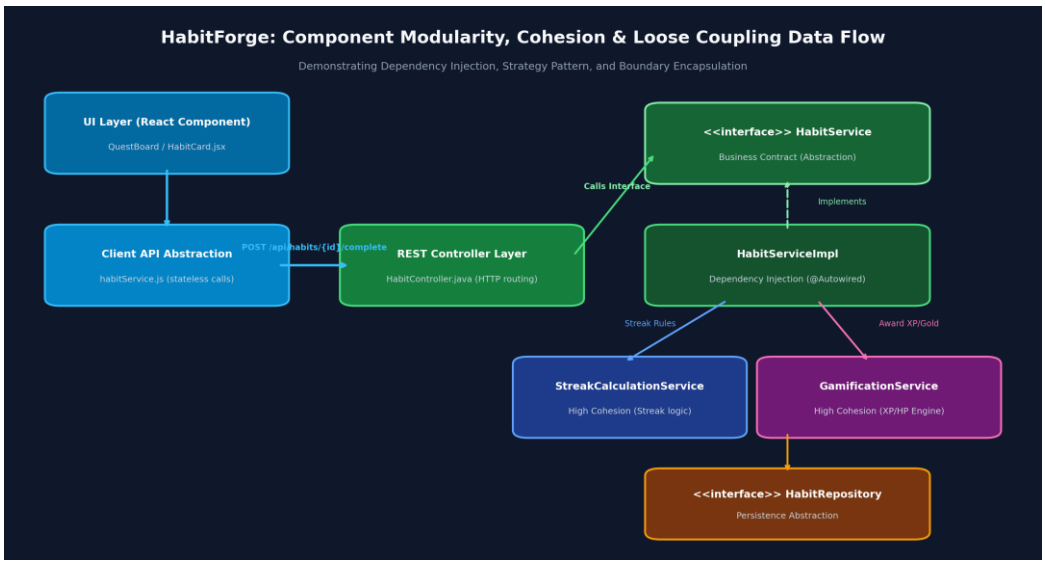


Figure 2: Component Modularity, Cohesion & Loose Coupling Data Flow

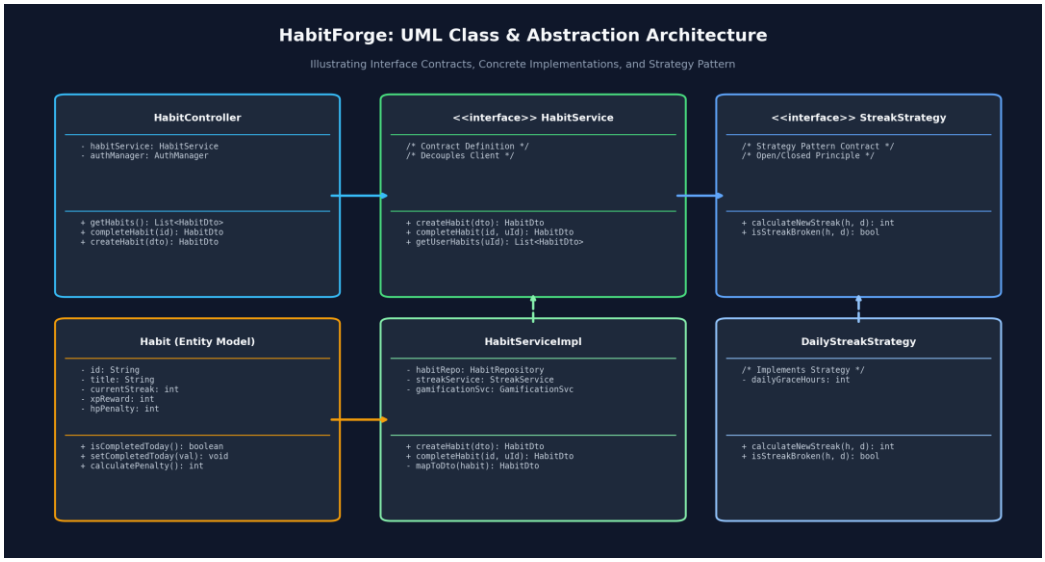
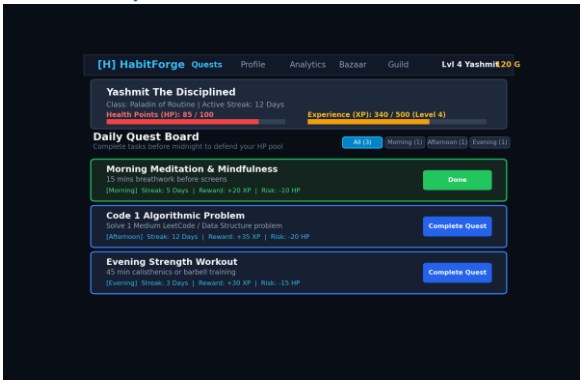


Figure 3: UML Class Diagram with Strategy Pattern & Repository Abstraction

c. User Interface Design (6 Figma Screens)

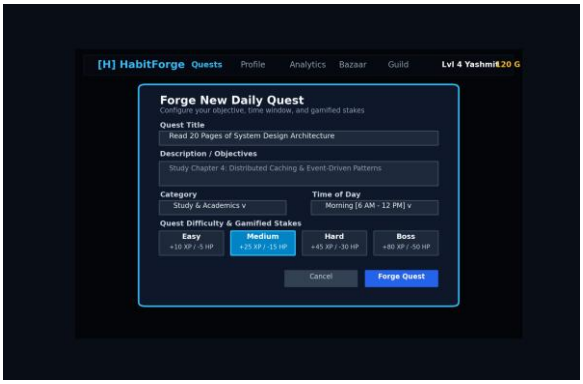
How Did You Make UI User-Friendly? The HabitForge interface converts wireframes from DA-1 into high-fidelity screens utilizing an Obsidian Dark RPG gaming visual language designed for cognitive comfort and behavioral reinforcement. Fitts's Law touch targets (>44px), cognitive chunking by time-of-day, and WCAG 2.1 AA compliant contrast (>7:1) are enforced globally. Prototype: https://www.figma.com/proto/habitforge-v2-system-design

Screen 1: Daily Quest Board



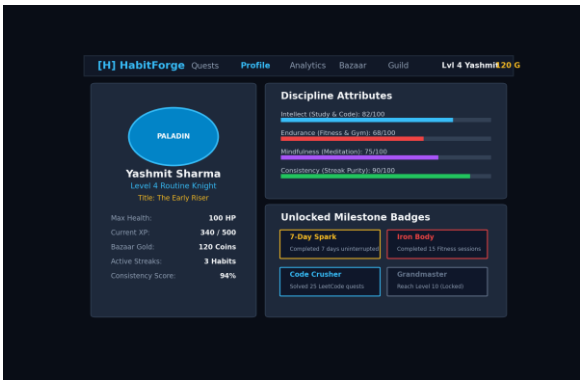
Animated HP/XP gauges, time-of-day habit chunking, and tactile checkoff buttons.

Screen 2: Quest Creator Modal



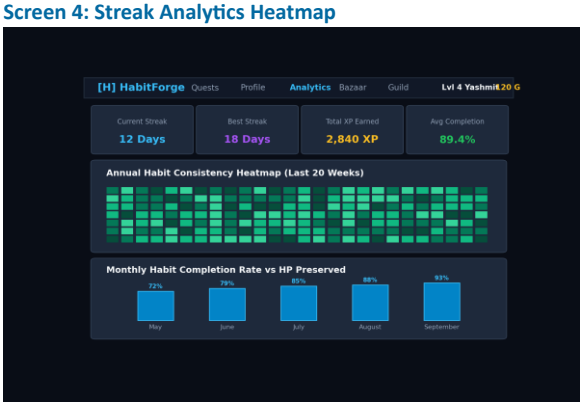
Progressive disclosure modal with difficulty tuning (Easy/Medium/Hard/Boss).

Screen 3: Hero Profile & Stats



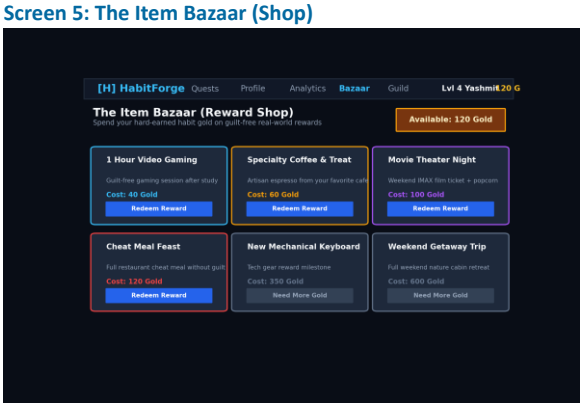
Character sheet with discipline attributes (Intellect, Endurance) and badges.

Screen 4: Streak Analytics Heatmap



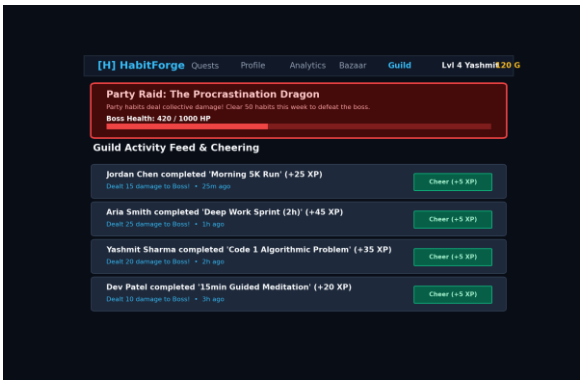
52-week GitHub-style emerald consistency heatmap and monthly bar charts.

Screen 5: The Item Bazaar (Shop)



Extrinsic rewards marketplace exchanging earned habit gold for guilt-free indulgences.

Screen 6: Adventurer's Guild Social



Party Boss Raids ('The Procrastination Dragon') and peer Cheers (+5 XP).

d. Design Decisions & Why (5 Key Architectural Choices)

1. React 18 SPA + Vite vs Multi-Page MVC: Selected React 18 SPA with Vite for sub-100ms client transitions and instant optimistic check-offs critical to the dopamine loop. Full-page reloads in MVC make animated progress bars feel sluggish.

2. Layered Spring Boot with DTOs vs Flat Controller-Entity: Enforces strict boundaries preventing database leakage and enabling unit testing. Flat binding of database models to HTTP endpoints creates critical security vulnerabilities.

3. Document NoSQL (MongoDB) vs Relational SQL: Selected MongoDB because habit entities feature dynamic, nested attributes (custom recurrence schedules, daily completion logs, badges) that map naturally to JSON documents.

4. Server-Authoritative Midnight Cron vs Client-Triggered Local Sync: Centralized Spring Boot @Scheduled cron task deducts HP and resets streaks at 23:59:59 server time. Client-side evaluation was rejected because users can manipulate device clocks to fake streaks.

5. Stateless JWT Authentication vs Stateful HTTP Sessions: Tokens are cryptographically verified per request, allowing the backend to scale horizontally without sticky server-side sessions.

Thought on Maintainability and Future Changes

? Strategy Pattern for Streaks: The StreakStrategy interface guarantees the Open/Closed Principle?new streak rules (weekend grace periods, sick days) are introduced without modifying core services.
? Native Mobile Apps: Exposing clean stateless RESTful JSON contracts means native mobile apps in Flutter or React Native can consume identical /api/habits endpoints without backend refactoring.
? Persistence Decoupling: The HabitRepository abstraction allows swapping MongoDB for AWS DynamoDB if scale increases to millions of users.

2. GitHub Repo Updates & Deliverables Proof

GitHub Repo Link: https://github.com/yashmit2812/HabitForge

Figma Prototype Link: https://www.figma.com/proto/habitforge-v2-system-design

a. Directory Structure (/docs/design/ and /design/)

```
HabitForge/
??? README.md                                    [Updated with Software Design DA-2 Section]
??? HabitForge_Software_Design_Document_DA2.docx    [Editable Word Document Source]
??? HabitForge_Software_Design_Document_DA2.pdf     [PDF Exported from Word]
??? docker-compose.yml                           [Multi-container orchestration]
??? docs/design/                                 (and mirrored in /design/)
?   ??? HabitForge_Software_Design_Document_DA2.docx
?   ??? HabitForge_Software_Design_Document_DA2.pdf
?   ??? habitforge_architecture.drawio           [Editable Draw.io XML Source]
?   ??? habitforge_architecture.png              [High-Res System Architecture PNG]
?   ??? habitforge_component_flow.drawio         [Editable Component Modularity XML]
?   ??? habitforge_component_flow.png            [High-Res Component Flow PNG]
?   ??? habitforge_class_diagram.drawio          [Editable UML Class Diagram XML]
?   ??? habitforge_class_diagram.png             [High-Res UML Class Diagram PNG]
?   ??? FIGMA_PROTOTYPE.md                       [Prototype Link, Tokens & Heuristics]
?   ??? screens/
?       ??? screen1_quest_board.png              [Figma Screen 1: Quest Board]
?       ??? screen2_create_habit.png             [Figma Screen 2: Quest Creator]
?       ??? screen3_hero_profile.png             [Figma Screen 3: Hero Profile]
?       ??? screen4_analytics_heatmap.png        [Figma Screen 4: Heatmap & KPIs]
?       ??? screen5_reward_shop.png              [Figma Screen 5: Item Bazaar]
?       ??? screen6_guild_social.png             [Figma Screen 6: Social Guild]
??? backend/src/main/java/com/example/HabitForge/  [Modular Java Spring Boot Codebase]
??? frontend/src/components/ & services/         [Modular React Components & Services]
```

b. Verification Table

Deliverable Item	Requirement Spec	Repository Location
Software Design Document	Word (.docx) & PDF (6-10 pages)	HabitForge_Software_Design_Document_DA2.docx / .pdf
Draw.io Diagrams	Editable source .drawio & PNGs	docs/design/habitforge_*.drawio & *.png
6 Figma UI Screens	High-fidelity mockups & UX specs	docs/design/screens/screen[1-6].png & FIGMA_PROTOTYPE.md
README.md Updates	Section 'Software Design'	README.md (with embedded diagrams & design summary)
Applied Principles	Abstraction, Modularity, Cohesion, Coupling	backend/src/... & frontend/src/... code implementations